PROJECT REPORT

Redis Caching and Cache Invalidation

We made a website that remembers answers from NASA so it does not have to ask NASA again and again, and that forgets the answer at the right time so it never shows old information.

Topic	6: Redis Caching and Cache Invalidation
Live website	https://imap-project-chi.vercel.app/
Code (GitHub)	https://github.com/vihaang123/imap_project
College	MPSTME, SVKM's NMIMS, Mumbai
Date	24 October 2026

Team

Member	Roll no.	What this person did
Vihaan Gandhi	E002	Backend and caching
Jash Visaria	E003	Invalidation (clearing old data) and the database part
Calvin D'souza	E013	Speed tests, automatic tests and documents
Arnav Jha	E009	Dashboard (the web page) and putting it online

1. What is this project about?

NASA publishes one space picture every day. This is called the **Astronomy Picture of the Day**, or **APOD**. Our website shows that picture. To get it, our server has to ask NASA's server over the internet. NASA is slow to answer: on our live website one request took **2375 milliseconds** (a bit more than 2 seconds). NASA also allows only a limited number of requests.

If 100 people open the same day's picture, we do not want to ask NASA 100 times for the same thing. So we use a **cache**.

> **In simple words:** A cache is a quick notebook. The first time, we ask NASA (slow) and write the answer in the notebook. Next time, we read the notebook (very fast) and do not bother NASA.

The second half of the project is the harder one: the notebook must not show **old** information. If the real information changes, the old note must be thrown away. This is called **cache invalidation**.

> **In simple words:** Invalidation means: when the real data changes, cross out the old note in the notebook so nobody reads wrong information.

What we set out to do

- Store NASA's answers in **Redis** (a very fast memory database) and show the difference between a slow first request and a fast repeat request.
- Try the two ways Django offers to cache: *low-level* (we control exactly what is saved) and *per-view* (Django saves the whole page).
- Remove old data in three ways: automatically, by button, and by a timer.
- Measure how much faster the website becomes, without sending too many requests to NASA.
- Put the working project online using only free services.

2. What did we use, and why?

Tool	What it is	Why we used it
Python and Django	A programming language and a website-building toolkit	Django already has a caching system and "signals" that tell us when data changes
Redis	A database that keeps data in memory (RAM), so it is extremely fast	It is the standard cache. It can also delete entries by itself after a set time
Upstash	A company that hosts Redis on the internet for free	Our website runs online, so it needs a Redis that is also online
django-redis	A small add-on that connects Django to Redis	Gives extra tools such as deleting many keys at once
NASA APOD API	NASA's public service that gives the daily picture	It is the slow service we are speeding up
SQLite / Postgres	Normal databases that store data on disk	Store the titles that a user edits (the data that can change)
HTML, CSS, JavaScript	The languages of web pages	Used for our dashboard, the control page of the project
Docker	A tool that runs software in a ready-made box	Starts Redis on a laptop with one command
Vercel	A free website hosting service	Runs our website online
GitHub	A website that stores code	Keeps our code, and every upload updates the live site

3. How the system works

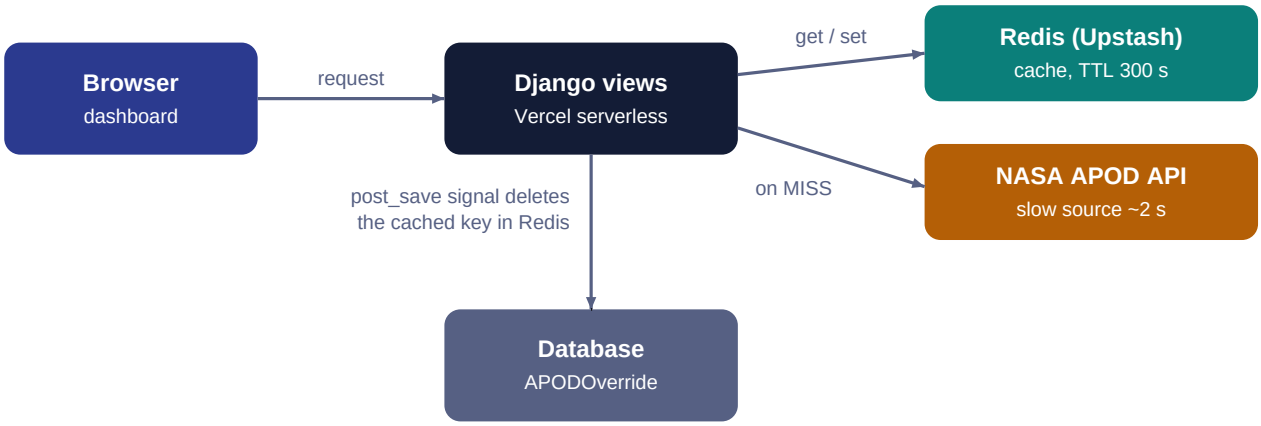


Figure 1. The browser talks to Django. Django checks Redis first. If the answer is not there, it asks NASA, saves the answer in Redis and replies. If someone edits the data, the old saved copy is deleted.

The four situations

Situation	What happens	Label shown
First request for a date (miss)	Redis has nothing. Django asks NASA (slow), saves the answer for 300 seconds, and replies.	MISS
Same date again (hit)	Redis already has the answer. Django replies at once and does not call NASA.	HIT
Someone edits the data	A signal deletes the saved copy. The next request is a MISS and returns the new data.	MISS
Uncached endpoint	Never uses the cache. Always asks NASA. We use it for comparison.	BYPASS

In simple words: HIT = the answer was found in the notebook. MISS = it was not found, so we had to ask NASA. HIT is fast, MISS is slow.

Important words

Word	Meaning
Cache	A fast temporary store of answers we already fetched
Hit / Miss	Found in the cache / not found in the cache
TTL (time to live)	How long a saved answer stays before it is deleted automatically. Ours is 300 seconds (5 minutes)
Key	The label on a saved answer. Ours is one per date, for example apod:2024-06-01
Invalidation	Deleting a saved answer because the real data has changed
Signal (Django)	An alarm that rings automatically when something is saved or deleted in the database
Hit ratio	Hits divided by all requests. Higher means the cache is helping more

4. How we built it

Pages and requests (endpoints)

Address	What it does
/	The dashboard (the control page)
/api/apod/uncached/	Asks NASA every time. Used as the slow baseline
/api/apod/low-level/	Uses the cache that we control ourselves
/api/apod/page-cached/	Uses Django's ready-made page cache
/api/cache/stats/	Shows hits, misses, hit ratio and time left
/api/cache/invalidate/	Deletes the saved copy for one date or for everything
/api/cache/stats/reset/	Sets the counters back to zero
/api/override/	Saves an edited title, which clears the old copy

How the cache works (cache-aside)

- **Step 1:** look in Redis for the date.
- **Step 2:** if it is there, return it (HIT).
- **Step 3:** if not, ask NASA, save the answer in Redis for 300 seconds, return it (MISS).
- We use one key per date so we can delete exactly one date when needed.

- **We never save errors.** If NASA fails once, we do not want everyone to see that failure.
- Hit and miss counters are kept in the cache, so every server instance shares the same numbers.

In simple words: It is like a student keeping notes: look at the notes first, and only open the textbook (NASA) if the answer is not in the notes.

How old data is removed (invalidation)

- **Automatic:** when a title is edited or deleted, a Django signal runs and deletes that date's saved copy. It works whether the edit comes from the dashboard, the admin page or code.
- **By button:** the dashboard can clear one date or everything.
- **By timer:** after 300 seconds Redis deletes the entry anyway. This is our safety net.
- Django's page cache saves entries under long keys made from the whole URL, so we cannot guess the key for one date. We delete those by matching a pattern. This is safe, but it removes slightly more than needed.

The dashboard

The dashboard is one web page that lets you try everything: a big number showing the time of the last request with a HIT or MISS label, a choice of endpoint, a date box, a mini benchmark button, buttons to clear the cache, a form to edit the title, a list of past requests with bars, the NASA picture, and a panel with the hit ratio and a countdown showing how long the saved answer will last.

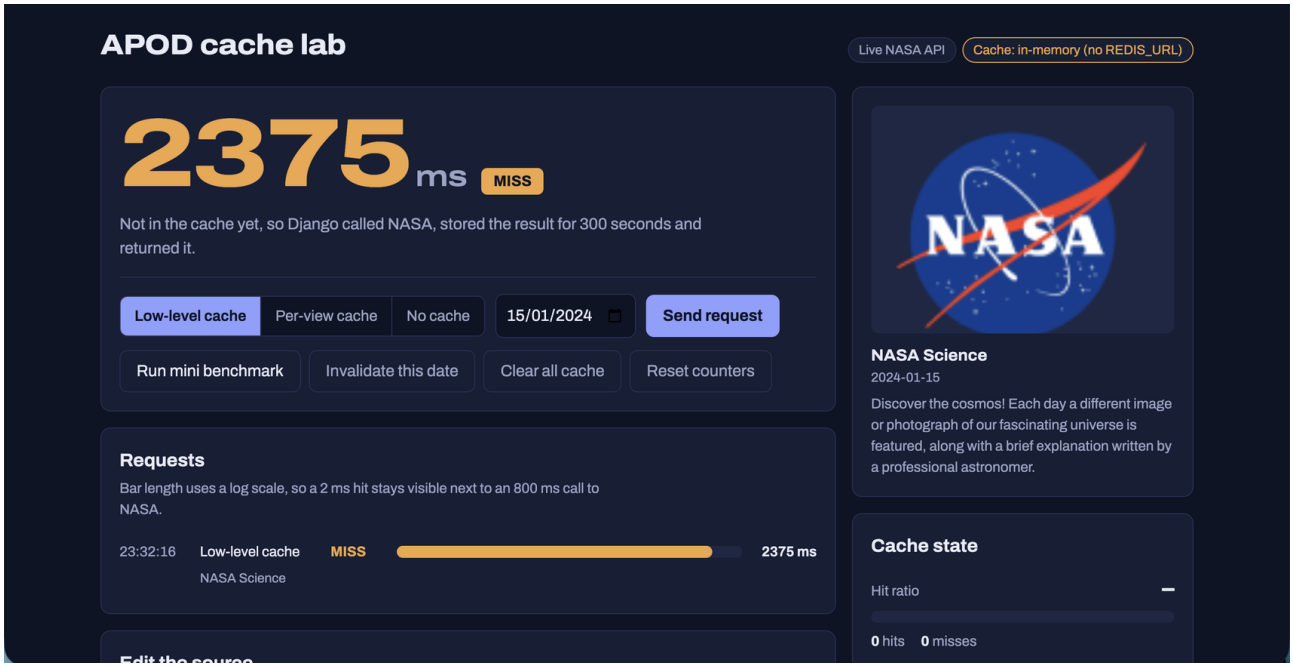


Figure 2. The live dashboard. The first request was a MISS and took 2375 ms because Django asked NASA. This screenshot was taken before Redis was connected, so its badge says in-memory; the live site now says Cache: Redis.

5. Putting it online

The code is on GitHub, and Vercel runs it. Every time we upload new code to GitHub, Vercel updates the live website automatically.

Vercel runs our code in short-lived "serverless" boxes that start and stop all the time. If each box kept its own private notebook, they would not share information. So we connected **Upstash Redis** (free plan). Now every box reads and writes the same cache.

In simple words: Without shared Redis, each Vercel box had its own memory and the hit counter kept showing zero. With Redis, all boxes share one cache, so a HIT works properly.

Keeping secrets safe

- The NASA key, Django secret key and Redis address are stored as secret settings in Vercel, never in the code.
- On a laptop they live in a file called `.env`, which GitHub ignores. The file `.env.example` shows only the names.
- The live site runs with debug mode off.

6. Testing and results

Automatic tests

We wrote **12 automatic tests**. They need no Redis and no NASA key because they use a fake memory and a fake NASA. They check that: a second request does not call NASA; the uncached endpoint always calls NASA; editing or deleting data clears the cache; the clear button works; wrong dates give an error; the counters add up; errors are never saved; HIT and MISS labels are correct; and the dashboard opens.

Test of the live website (24 October 2026)

What we checked	What we saw
Latest version online	Ready, created after Redis was connected
Badge on the dashboard	Cache: Redis (earlier it said in-memory)
First request for 2024-06-01	MISS: it asked NASA
Same request again	HIT: it came from Redis
Counters afterwards	1 hit, 1 miss, ratio 0.5, 295 of 300 seconds left. They were read by a different request, which proves the cache is shared

Speed

Measurement	Result	Where it came from
Request that asks NASA (MISS)	2375 ms	Our live website, see Figure 2
Request answered from cache (HIT)	Add from the dashboard benchmark	Dashboard: Run mini benchmark
How many times faster	MISS time divided by HIT time	Dashboard: Run mini benchmark

We checked the logic once on a laptop with a fake NASA that waits 0.8 seconds. Hits took 2 to 9 ms and misses about 850 ms. That tests our code and Redis, not NASA, so we do not treat it as the project result.

7. Questions from the assignment

Q1. What is a cache hit and what is a cache miss?

A **hit** means the answer is already saved, so we return it straight away. A **miss** means it is not saved (never fetched, expired, or deleted), so we must do the slow work: ask NASA, save the answer, and then return it. The first request for any date is always a miss. The **hit ratio** is hits divided by total requests. A high ratio means the cache is doing its job, and our stats page shows it.

Q2. What is TTL, and why is invalidation hard?

TTL is the number of seconds a saved answer is kept before Redis deletes it by itself. Ours is 300 seconds. It is a safety net, because even if we forget to clear something it cannot stay wrong for ever. A short TTL gives fresher data but more misses. A long TTL is faster but may show older data.

Invalidation is hard because the cache is only a copy, and the real data can change without the cache knowing. We have to find every copy of that data and delete all of them. In our project the same date can be stored in two places (our own cache and Django's page cache), and the page cache uses long keys that we cannot rebuild for one date, so we delete by pattern. There is also a timing problem: a request can put old data back into the cache just as we delete it. And we must balance deleting too much (slower) against deleting too little (old data).

> **In simple words:** Making a copy is easy. Knowing exactly when and which copies are now wrong is the hard part.

Q3. What is the difference between per-view caching and low-level caching?

Per-view caching (`@cache_page`) saves the whole web response for a URL. It needs one line of code, but we cannot choose the key and it is awkward to clear just one date. **Low-level caching** (`cache.get` and `cache.set`) lets us choose the key, the time and exactly what to save. It needs more code but allows clearing one date, counting hits and misses, and reusing the data elsewhere. We built both and compared them.

	Per-view cache	Low-level cache
Code needed	One line	More code
What is saved	The whole response	Only the data we choose
Key	Made by Django from the URL	Chosen by us (one per date)
Clearing one date	Awkward (pattern delete)	Easy and exact
Best for	Simple pages that are the same for everyone	Data we need to control precisely

8. Rate limits and safety

- NASA limits how often we may call it, so we never load test it. The dashboard benchmark makes only 2 NASA calls and the script about 6. The rest is served from our own cache.
- Keys and passwords are never put in the code.
- The edit and clear endpoints have no login. That is fine for a demo, but a real system would need one.

9. Limits of our project and what we could add

- **Cache stampede:** if many people ask for a missing date at the same moment, all of them call NASA. A lock would fix that.

- If Redis stops working, requests currently fail. We could treat that as a miss instead.
- Replace pattern deleting with versioned keys.
- Count hits for the per-view endpoint too.
- Use a longer TTL for old dates, because past pictures never change.
- Add a login to the edit and clear endpoints.

10. Who did what

Member	Work done	Main files
Vihaan Gandhi (E002)	Backend and caching: the cache-aside logic, both caching endpoints, HIT and MISS labels, cache settings	apod/views.py, apod/services.py, config/settings.py
Jash Visaria (E003)	Invalidation and data: the edit model, signals, clearing functions, admin page	apod/models.py, signals.py, apps.py, cache_utils.py, admin.py
Calvin D'souza (E013)	Testing and documents: benchmark script, 12 tests, README and this report	scripts/benchmark.py, apod/tests.py, config/test_settings.py
Arnav Jha (E009)	Dashboard and online deployment: web page, Vercel setup, Docker, settings template	dashboard.html, api/index.py, vercel.json, docker-compose.yml

11. References

- Django documentation, The cache framework: https://docs.djangoproject.com/en/stable/topics/cache/
- django-redis: https://github.com/jazzband/django-redis
- Redis documentation: https://redis.io/docs/
- NASA open APIs: https://api.nasa.gov/